Algorithmics and Data Structures

Session 3
Searching and Sorting

Ali Ismail-Fawaz

ENSISA, Université Haute-Alsace

Overview

What We'll Cover Today:

Objectives:

- Understand the purpose of searching and sorting in algorithms
- Learn classic algorithms and their time complexity
- Analyze trade-offs in efficiency, stability, and use cases

Topics to Be Covered:

- **Search Algorithms**:
  - Linear Search
  - Binary Search
  - Jump Search
- **Sorting Algorithms**:
  - Selection Sort
  - Insertion Sort
  - Bubble Sort
  - Merge Sort
  - Quick Sort

Contents

Searching Algorithms

Simple Sorting Algorithms

Complex Sorting Algorithms

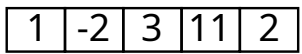
Linear Search

Overview:

- Checks each element in a list sequentially
- Works on both sorted and unsorted data
- Returns the index of the target if found, else -1

Example:

target: 11



Complexity:

- Best Case: $\mathcal{O}(1)$ (target is first)
- Worst Case: $\mathcal{O}(n)$ (target is last or missing)
- Space: $\mathcal{O}(1)$ (no extra memory)

When to Use:

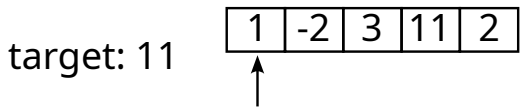
- Small or unsorted datasets
- You need a quick, simple implementation

Linear Search

Overview:

- Checks each element in a list sequentially
- Works on both sorted and unsorted data
- Returns the index of the target if found, else -1

Example:



Complexity:

- Best Case: $\mathcal{O}(1)$ (target is first)
- Worst Case: $\mathcal{O}(n)$ (target is last or missing)
- Space: $\mathcal{O}(1)$ (no extra memory)

When to Use:

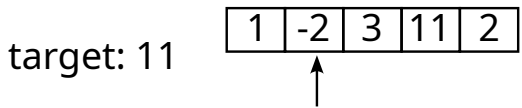
- Small or unsorted datasets
- You need a quick, simple implementation

Linear Search

Overview:

- Checks each element in a list sequentially
- Works on both sorted and unsorted data
- Returns the index of the target if found, else -1

Example:



Complexity:

- Best Case: $\mathcal{O}(1)$ (target is first)
- Worst Case: $\mathcal{O}(n)$ (target is last or missing)
- Space: $\mathcal{O}(1)$ (no extra memory)

When to Use:

- Small or unsorted datasets
- You need a quick, simple implementation

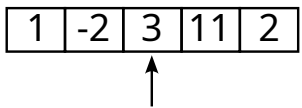
Linear Search

Overview:

- Checks each element in a list sequentially
- Works on both sorted and unsorted data
- Returns the index of the target if found, else -1

Example:

target: 11



Complexity:

- Best Case: $\mathcal{O}(1)$ (target is first)
- Worst Case: $\mathcal{O}(n)$ (target is last or missing)
- Space: $\mathcal{O}(1)$ (no extra memory)

When to Use:

- Small or unsorted datasets
- You need a quick, simple implementation

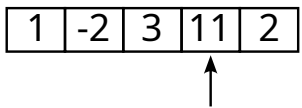
Linear Search

Overview:

- Checks each element in a list sequentially
- Works on both sorted and unsorted data
- Returns the index of the target if found, else -1

Example:

target: 11



Complexity:

- Best Case: $\mathcal{O}(1)$ (target is first)
- Worst Case: $\mathcal{O}(n)$ (target is last or missing)
- Space: $\mathcal{O}(1)$ (no extra memory)

When to Use:

- Small or unsorted datasets
- You need a quick, simple implementation

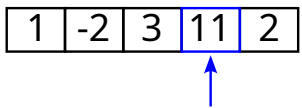
Linear Search

Overview:

- Checks each element in a list sequentially
- Works on both sorted and unsorted data
- Returns the index of the target if found, else -1

Example:

target: 11



Complexity:

- Best Case: $\mathcal{O}(1)$ (target is first)
- Worst Case: $\mathcal{O}(n)$ (target is last or missing)
- Space: $\mathcal{O}(1)$ (no extra memory)

When to Use:

- Small or unsorted datasets
- You need a quick, simple implementation

Jump Search

Idea:

- Works on a **sorted array**
- Instead of checking every element, jump ahead by a fixed number of steps
- Once you pass the target, do a linear search backward in the previous block
- Reduces the number of comparisons compared to linear search
- Example: Finding Your Favorite Snack on a Huge Shelf

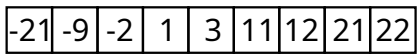
Jump Search

Idea:

- Works on a **sorted array**
- Instead of checking every element, jump ahead by a fixed number of steps
- Once you pass the target, do a linear search backward in the previous block
- Reduces the number of comparisons compared to linear search
- Example: Finding Your Favorite Snack on a Huge Shelf

target: 21

size = 9 jump = 3



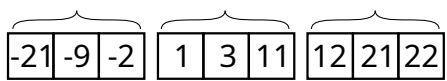
Jump Search

Idea:

- Works on a **sorted array**
- Instead of checking every element, jump ahead by a fixed number of steps
- Once you pass the target, do a linear search backward in the previous block
- Reduces the number of comparisons compared to linear search
- Example: Finding Your Favorite Snack on a Huge Shelf

target: 21

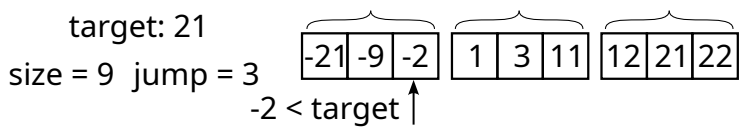
size = 9 jump = 3



Jump Search

Idea:

- Works on a **sorted array**
- Instead of checking every element, jump ahead by a fixed number of steps
- Once you pass the target, do a linear search backward in the previous block
- Reduces the number of comparisons compared to linear search
- Example: Finding Your Favorite Snack on a Huge Shelf



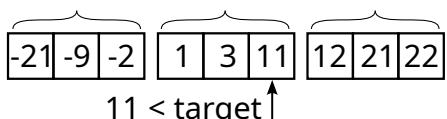
Jump Search

Idea:

- Works on a **sorted array**
- Instead of checking every element, jump ahead by a fixed number of steps
- Once you pass the target, do a linear search backward in the previous block
- Reduces the number of comparisons compared to linear search
- Example: Finding Your Favorite Snack on a Huge Shelf

target: 21

size = 9 jump = 3



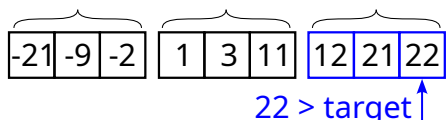
Jump Search

Idea:

- Works on a **sorted array**
- Instead of checking every element, jump ahead by a fixed number of steps
- Once you pass the target, do a linear search backward in the previous block
- Reduces the number of comparisons compared to linear search
- Example: Finding Your Favorite Snack on a Huge Shelf

target: 21

size = 9 jump = 3



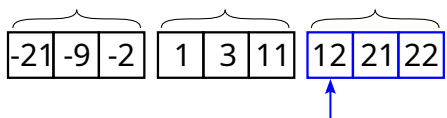
Jump Search

Idea:

- Works on a **sorted array**
- Instead of checking every element, jump ahead by a fixed number of steps
- Once you pass the target, do a linear search backward in the previous block
- Reduces the number of comparisons compared to linear search
- Example: Finding Your Favorite Snack on a Huge Shelf

target: 21

size = 9 jump = 3



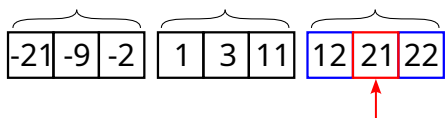
Jump Search

Idea:

- Works on a **sorted array**
- Instead of checking every element, jump ahead by a fixed number of steps
- Once you pass the target, do a linear search backward in the previous block
- Reduces the number of comparisons compared to linear search
- Example: Finding Your Favorite Snack on a Huge Shelf

target: 21

size = 9 jump = 3



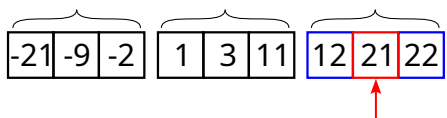
Jump Search

Idea:

- Works on a **sorted array**
- Instead of checking every element, jump ahead by a fixed number of steps
- Once you pass the target, do a linear search backward in the previous block
- Reduces the number of comparisons compared to linear search
- Example: Finding Your Favorite Snack on a Huge Shelf

target: 21

size = 9 jump = 3



Question for You:

- What is the best jump size to minimize the number of comparisons?
- Assume the array has n elements and the jump size is m

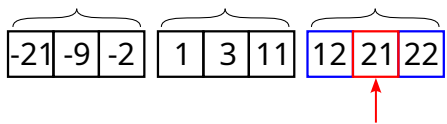
Jump Search

Idea:

- Works on a **sorted array**
- Instead of checking every element, jump ahead by a fixed number of steps
- Once you pass the target, do a linear search backward in the previous block
- Reduces the number of comparisons compared to linear search
- Example: Finding Your Favorite Snack on a Huge Shelf

target: 21

size = 9 jump = 3



Question for You:

- What is the best jump size to minimize the number of comparisons?
- Assume the array has n elements and the jump size is m

Hint – Total comparisons:

$$\frac{n}{m} + m$$

Can you choose m to minimize this expression?

Deriving the Optimal Jump Size

We want to minimize the total number of comparisons:

$$f(m) = \frac{n}{m} + m$$

Step 1: Take the derivative

$$f'(m) = -\frac{n}{m^2} + 1$$

Step 2: Set the derivative to 0

$$f'(m) = 0 \Rightarrow -\frac{n}{m^2} + 1 = 0$$

Step 3: Solve for m

$$\frac{n}{m^2} = 1 \Rightarrow m^2 = n \Rightarrow m = \sqrt{n}$$

Conclusion: The optimal jump size to minimize comparisons is:

$$\boxed{m = \sqrt{n}}$$

Time Complexity: $\mathcal{O}(\sqrt{n})$ **Space Complexity**: $\mathcal{O}(1)$

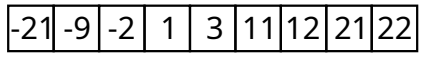
Binary Search

Definition:

- An efficient algorithm for searching in a **sorted array**
- Repeatedly divides the search interval in half
- Eliminates half of the remaining elements each time
- Example: Guessing a Number Between 1 and 100

target: 21

size = 9



How It Works:

- Start with two pointers: `low = 0`, `high = n - 1`
- While `low` $\leq$ `high`:
  - Compute `mid = (low + high) / 2`
  - If `target == arr[mid]`, return index
  - If `target < arr[mid]`, search left half
  - Else, search right half

Time Complexity: $\mathcal{O}(\log n)$ **Space Complexity**: $\mathcal{O}(1)$ (iterative) and $\mathcal{O}(\log n)$ (recursive)

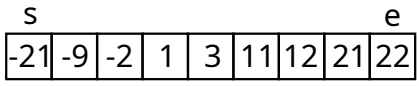
Binary Search

Definition:

- An efficient algorithm for searching in a **sorted array**
- Repeatedly divides the search interval in half
- Eliminates half of the remaining elements each time
- Example: Guessing a Number Between 1 and 100

target: 21

size = 9



How It Works:

- Start with two pointers: `low = 0`, `high = n - 1`
- While `low` $\leq$ `high`:
  - Compute `mid = (low + high) / 2`
  - If `target == arr[mid]`, return index
  - If `target < arr[mid]`, search left half
  - Else, search right half

Time Complexity: $\mathcal{O}(\log n)$ **Space Complexity**: $\mathcal{O}(1)$ (iterative) and $\mathcal{O}(\log n)$ (recursive)

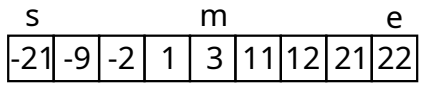
Binary Search

Definition:

- An efficient algorithm for searching in a **sorted array**
- Repeatedly divides the search interval in half
- Eliminates half of the remaining elements each time
- Example: Guessing a Number Between 1 and 100

target: 21

size = 9



How It Works:

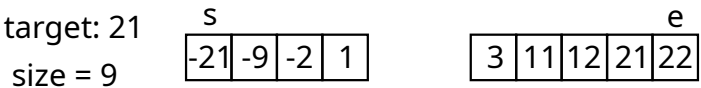
- Start with two pointers: `low = 0`, `high = n - 1`
- While `low` $\leq$ `high`:
  - Compute `mid = (low + high) / 2`
  - If `target == arr[mid]`, return index
  - If `target < arr[mid]`, search left half
  - Else, search right half

Time Complexity: $\mathcal{O}(\log n)$ **Space Complexity**: $\mathcal{O}(1)$ (iterative) and $\mathcal{O}(\log n)$ (recursive)

Binary Search

Definition:

- An efficient algorithm for searching in a **sorted array**
- Repeatedly divides the search interval in half
- Eliminates half of the remaining elements each time
- Example: Guessing a Number Between 1 and 100



How It Works:

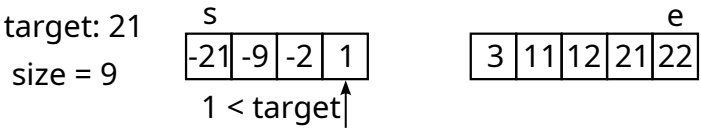
- Start with two pointers: `low = 0`, `high = n - 1`
- While `low` $\leq$ `high`:
  - Compute `mid = (low + high) / 2`
  - If `target == arr[mid]`, return index
  - If `target < arr[mid]`, search left half
  - Else, search right half

Time Complexity: $\mathcal{O}(\log n)$ **Space Complexity**: $\mathcal{O}(1)$ (iterative) and $\mathcal{O}(\log n)$ (recursive)

Binary Search

Definition:

- An efficient algorithm for searching in a **sorted array**
- Repeatedly divides the search interval in half
- Eliminates half of the remaining elements each time
- Example: Guessing a Number Between 1 and 100



How It Works:

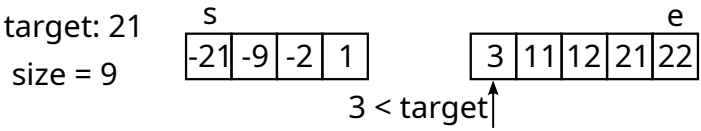
- Start with two pointers: `low = 0`, `high = n - 1`
- While `low` $\leq$ `high`:
  - Compute `mid = (low + high) / 2`
  - If `target == arr[mid]`, return index
  - If `target < arr[mid]`, search left half
  - Else, search right half

Time Complexity: $\mathcal{O}(\log n)$ **Space Complexity**: $\mathcal{O}(1)$ (iterative) and $\mathcal{O}(\log n)$ (recursive)

Binary Search

Definition:

- An efficient algorithm for searching in a **sorted array**
- Repeatedly divides the search interval in half
- Eliminates half of the remaining elements each time
- Example: Guessing a Number Between 1 and 100



How It Works:

- Start with two pointers: `low = 0`, `high = n - 1`
- While `low` $\leq$ `high`:
  - Compute `mid = (low + high) / 2`
  - If `target == arr[mid]`, return index
  - If `target < arr[mid]`, search left half
  - Else, search right half

Time Complexity: $\mathcal{O}(\log n)$ **Space Complexity**: $\mathcal{O}(1)$ (iterative) and $\mathcal{O}(\log n)$ (recursive)

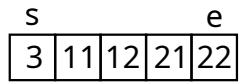
Binary Search

Definition:

- An efficient algorithm for searching in a **sorted array**
- Repeatedly divides the search interval in half
- Eliminates half of the remaining elements each time
- Example: Guessing a Number Between 1 and 100

target: 21

size = 9



How It Works:

- Start with two pointers: `low = 0`, `high = n - 1`
- While `low` $\leq$ `high`:
  - Compute `mid = (low + high) / 2`
  - If `target == arr[mid]`, return index
  - If `target < arr[mid]`, search left half
  - Else, search right half

Time Complexity: $\mathcal{O}(\log n)$ **Space Complexity**: $\mathcal{O}(1)$ (iterative) and $\mathcal{O}(\log n)$ (recursive)

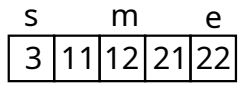
Binary Search

Definition:

- An efficient algorithm for searching in a **sorted array**
- Repeatedly divides the search interval in half
- Eliminates half of the remaining elements each time
- Example: Guessing a Number Between 1 and 100

target: 21

size = 9



How It Works:

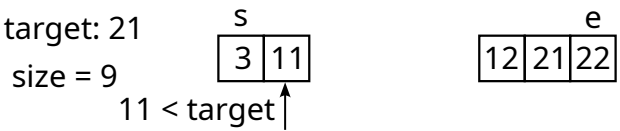
- Start with two pointers: `low = 0`, `high = n - 1`
- While `low` $\leq$ `high`:
  - Compute `mid = (low + high) / 2`
  - If `target == arr[mid]`, return index
  - If `target < arr[mid]`, search left half
  - Else, search right half

Time Complexity: $\mathcal{O}(\log n)$ **Space Complexity**: $\mathcal{O}(1)$ (iterative) and $\mathcal{O}(\log n)$ (recursive)

Binary Search

Definition:

- An efficient algorithm for searching in a **sorted array**
- Repeatedly divides the search interval in half
- Eliminates half of the remaining elements each time
- Example: Guessing a Number Between 1 and 100



How It Works:

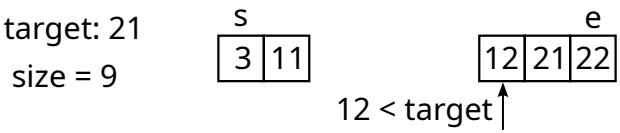
- Start with two pointers: `low = 0`, `high = n - 1`
- While `low` $\leq$ `high`:
  - Compute `mid = (low + high) / 2`
  - If `target == arr[mid]`, return index
  - If `target < arr[mid]`, search left half
  - Else, search right half

Time Complexity: $\mathcal{O}(\log n)$ **Space Complexity**: $\mathcal{O}(1)$ (iterative) and $\mathcal{O}(\log n)$ (recursive)

Binary Search

Definition:

- An efficient algorithm for searching in a **sorted array**
- Repeatedly divides the search interval in half
- Eliminates half of the remaining elements each time
- Example: Guessing a Number Between 1 and 100



How It Works:

- Start with two pointers: `low = 0`, `high = n - 1`
- While `low` $\leq$ `high`:
  - Compute `mid = (low + high) / 2`
  - If `target == arr[mid]`, return index
  - If `target < arr[mid]`, search left half
  - Else, search right half

Time Complexity: $\mathcal{O}(\log n)$ **Space Complexity**: $\mathcal{O}(1)$ (iterative) and $\mathcal{O}(\log n)$ (recursive)

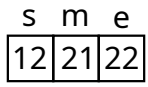
Binary Search

Definition:

- An efficient algorithm for searching in a **sorted array**
- Repeatedly divides the search interval in half
- Eliminates half of the remaining elements each time
- Example: Guessing a Number Between 1 and 100

target: 21

size = 9



How It Works:

- Start with two pointers: `low = 0`, `high = n - 1`
- While `low` $\leq$ `high`:
  - Compute `mid = (low + high) / 2`
  - If `target == arr[mid]`, return index
  - If `target < arr[mid]`, search left half
  - Else, search right half

Time Complexity: $\mathcal{O}(\log n)$ **Space Complexity**: $\mathcal{O}(1)$ (iterative) and $\mathcal{O}(\log n)$ (recursive)

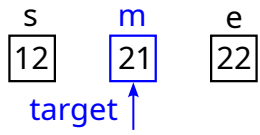
Binary Search

Definition:

- An efficient algorithm for searching in a **sorted array**
- Repeatedly divides the search interval in half
- Eliminates half of the remaining elements each time
- Example: Guessing a Number Between 1 and 100

target: 21

size = 9



How It Works:

- Start with two pointers: `low = 0`, `high = n - 1`
- While `low` $\leq$ `high`:
  - Compute `mid = (low + high) / 2`
  - If `target == arr[mid]`, return index
  - If `target < arr[mid]`, search left half
  - Else, search right half

Time Complexity: $\mathcal{O}(\log n)$ **Space Complexity**: $\mathcal{O}(1)$ (iterative) and $\mathcal{O}(\log n)$ (recursive)

Jump Search vs Binary Search (Pseudocode)

Jump Search

Input: sorted list A, target x
Output: index of x or -1
Instructions:

1. Let $n \leftarrow$ length of A
2. Let $m \leftarrow$ jump size
3. Set $prev \leftarrow 0$
4. While $A[\min(m, n) - 1] < x$:
   a. $prev \leftarrow m$
   b. $m \leftarrow m +$ **jump size**
   c. **If** $prev \geq n$, **return** -1
5. For i from $prev$ to $\min(m, n)$:
   a. **If** $A[i] == x$, **return** i
6. Return -1

Binary Search

Input: sorted array A, target x
Output: index of x or -1
Instructions:

1. Set $low \leftarrow 0$, $high \leftarrow n - 1$
2. While $low \leq high$:
   a. $mid \leftarrow (low + high) \div 2$
   b. **If** $A[mid] == x$, **return** mid
   c. **If** $A[mid] < x$, $low \leftarrow mid + 1$
   d. **Else**, $high \leftarrow mid - 1$
3. Return -1

Contents

Searching Algorithms

Simple Sorting Algorithms

Complex Sorting Algorithms

Selection Sort

Idea:

- Repeatedly find the **smallest element** from the unsorted part
- Swap it with the first unsorted element
- Shrinks the unsorted part one step at a time

Example (sorting [4, 2, 5, 1]):

- Step 1: smallest is 1 $\rightarrow$ swap with 4 $\rightarrow$ [1, 2, 5, 4]
- Step 2: smallest is 2 $\rightarrow$ already in place
- Step 3: smallest is 4 $\rightarrow$ swap with 5 $\rightarrow$ [1, 2, 4, 5]

Complexity:

- Time: $O(n^2)$
- Space: $O(1)$

Pros and Cons:

- Simple, easy to implement
- Not efficient for large lists

Selection Sort – Pseudocode

Selection Sort

Input: array A of length n
Output: sorted array A in ascending order
Instructions:

1. For i from 0 to $n-2$:
   a. $min \leftarrow i$
   b. For j from $i+1$ to $n-1$:
      i. If $A[j] < A[min]$, then $min \leftarrow j$
   c. Swap $A[i]$ and $A[min]$
2. Return A

Insertion Sort

Idea:

- Builds the sorted array **one element at a time**
- At each step, takes the next element and inserts it into its correct position among the already-sorted elements

Example (sorting [4, 2, 5, 1]):

- Step 1: [4, 2] $\rightarrow$ insert 2 before 4 $\rightarrow$ [2, 4, 5, 1]
- Step 2: [2, 4, 5] $\rightarrow$ 5 already in place
- Step 3: [2, 4, 5, 1] $\rightarrow$ insert 1 at start $\rightarrow$ [1, 2, 4, 5]

Complexity:

- Time: $O(n^2)$ in worst case, $O(n)$ in best case (nearly sorted)
- Space: $O(1)$

Pros and Cons:

- Efficient for small or nearly sorted arrays
- Simple and intuitive

Insertion Sort – Pseudocode

Insertion Sort

Input: array A of length n
Output: sorted array A in ascending order
Instructions:

1. For i from 1 to $n-1$:
   a. $key \leftarrow A[i]$
   b. $j \leftarrow i-1$
   c. While $j \geq 0$ and $A[j] > key$:
      i. $A[j+1] \leftarrow A[j]$
      ii. $j \leftarrow j-1$
   d. $A[j+1] \leftarrow key$
2. Return A

Bubble Sort

Idea:

- Repeatedly step through the list
- Compare adjacent elements and **swap them** if they are in the wrong order
- Each pass "bubbles up" the largest unsorted element to its correct position

Example (sorting [4, 2, 5, 1]):

- Pass 1: [4, 2, 5, 1] $\rightarrow$ [2, 4, 1, 5]
- Pass 2: [2, 4, 1, 5] $\rightarrow$ [2, 1, 4, 5]
- Pass 3: [2, 1, 4, 5] $\rightarrow$ [1, 2, 4, 5]

Complexity:

- Time: $O(n^2)$ worst and average
- Space: $O(1)$

Pros and Cons:

- Very simple, but inefficient on large lists
- Good for educational purposes

Contents

Searching Algorithms

Simple Sorting Algorithms

Complex Sorting Algorithms

Merge Sort

Idea:

- A classic **Divide and Conquer** sorting algorithm
- Recursively divides the list into two halves
- Sorts each half, then **merges** them into a single sorted list

How It Works:

- Divide: Split the array into left and right halves
- Conquer: Recursively sort each half
- Combine: Merge the two sorted halves

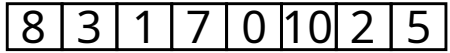
Complexity:

- Time: $O(n \log n)$ in all cases (best, average, worst)
- Space: $O(n)$ extra memory for merging

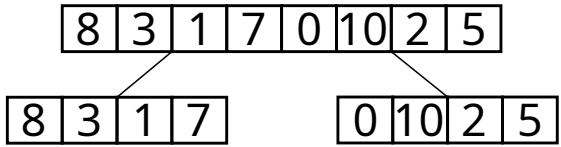
Pros and Cons:

- Good for large datasets and linked lists
- Requires extra space $\rightarrow$ not in-place

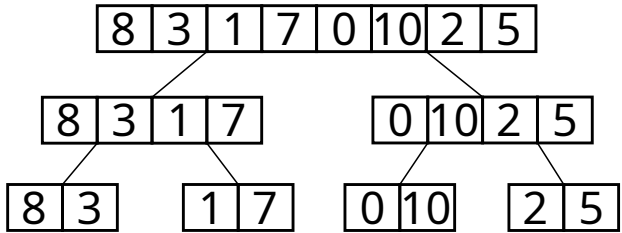
Merge Sort Example



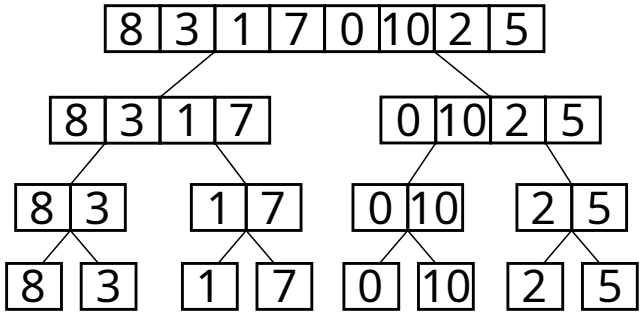
Merge Sort Example



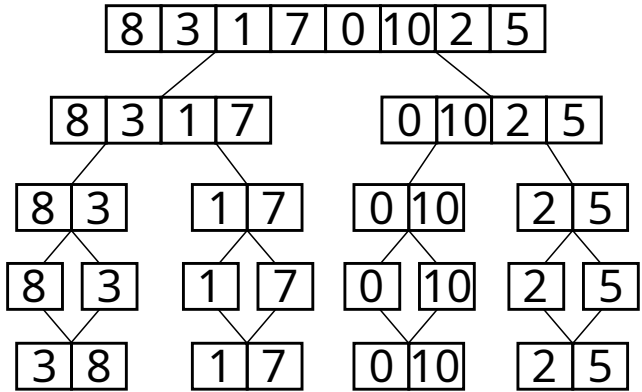
Merge Sort Example



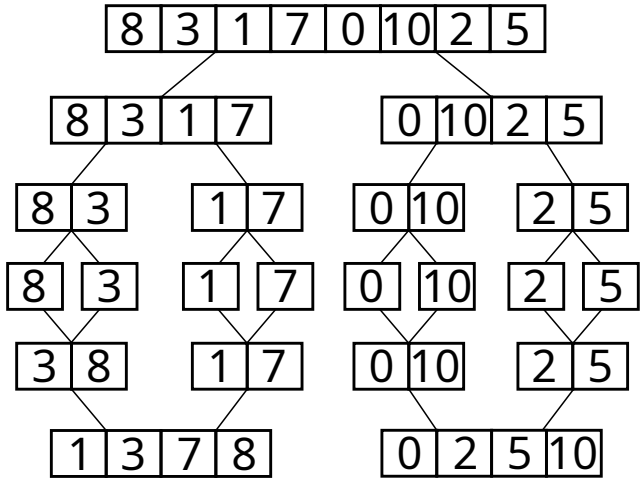
Merge Sort Example



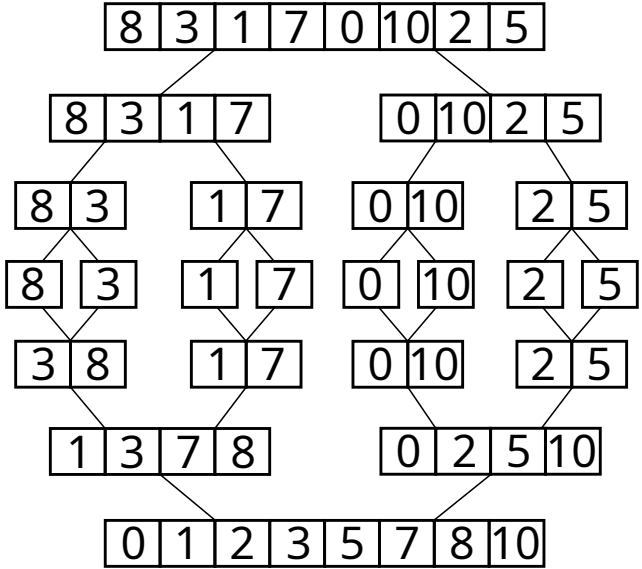
Merge Sort Example



Merge Sort Example



Merge Sort Example



Merge Sort Pseudocode

Recursive Merge Sort

Input: array A of length n
Output: sorted array A in ascending order
Instructions:

1. If $n \leq 1$, return A
2. $mid \leftarrow n \div 2$
3. $L \leftarrow$ MergeSort($A[0 : mid - 1]$)
4. $R \leftarrow$ MergeSort($A[mid : n - 1]$)
5. Return Merge(L, R)

Note: Merge(L, R) returns a sorted combination of L and R

Quick Sort

Idea:

- A **Divide and Conquer** algorithm like Merge Sort
- Picks a **pivot**, partitions the list into two:
  - Elements less than pivot
  - Elements greater than pivot
- Recursively applies the same strategy to both parts

How It Works (example with pivot = last element):

- Select pivot from array (e.g., last element)
- Rearrange so all smaller elements go to the left, larger to the right
- Place pivot in its correct position
- Recurse on left and right subarrays

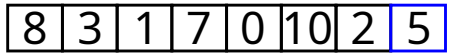
Complexity:

- Best/Average: $O(n \log n)$
- Space: $O(\log n)$ (due to recursion stack, in-place)

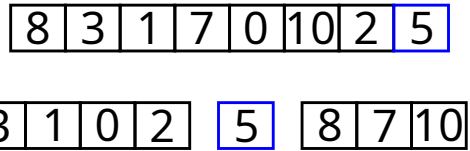
Pros and Cons:

- Very fast in practice (better constants than Merge Sort)
- Not stable; worst case can be quadratic

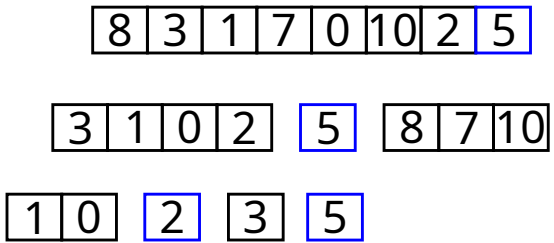
Quick Sort Example



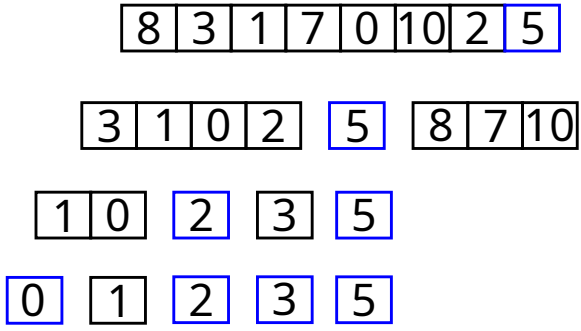
Quick Sort Example



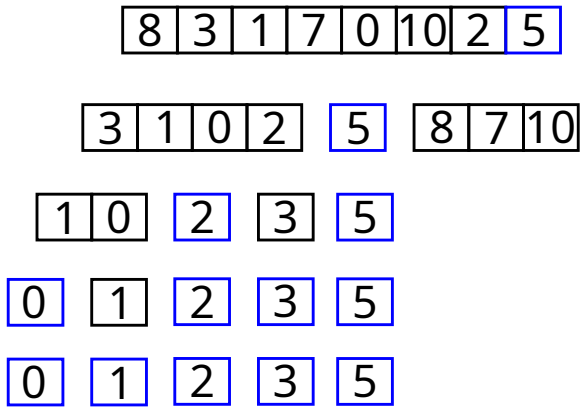
Quick Sort Example



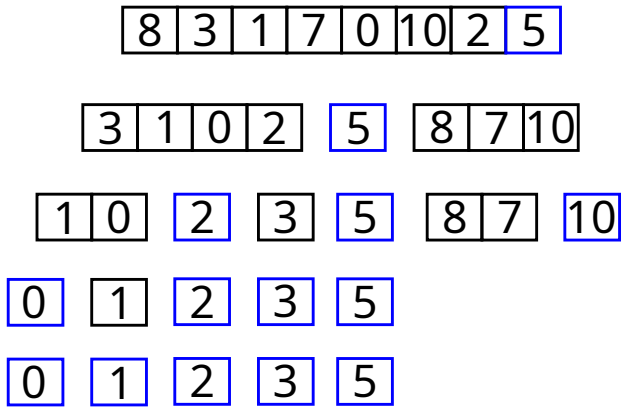
Quick Sort Example



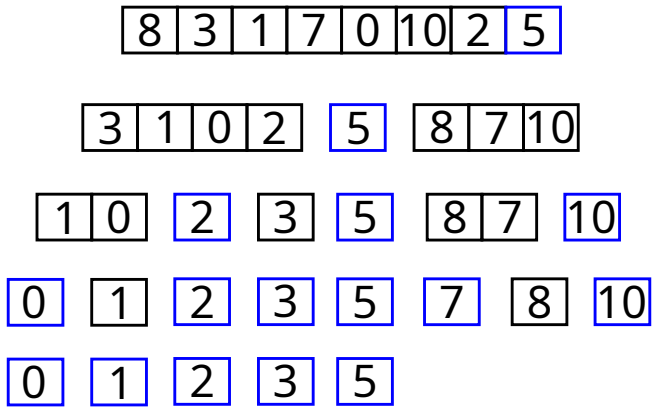
Quick Sort Example



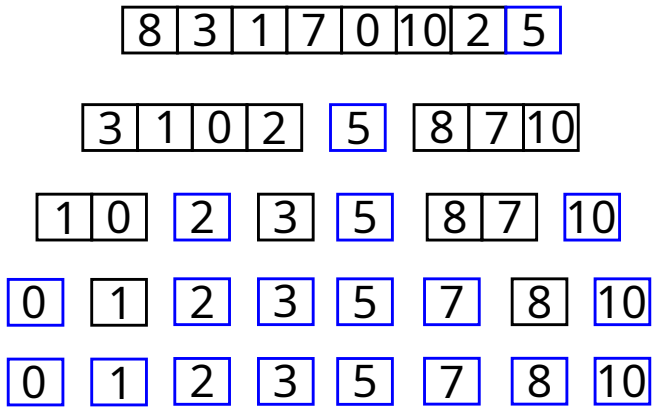
Quick Sort Example



Quick Sort Example



Quick Sort Example



Quick Sort – Pseudocode

Quick Sort

Input: list A, indices low, $high$
Output: sorted list A in ascending order
Instructions:

1. If $low < high$:
   a. $p \leftarrow$ Partition(A, low, $high$)
   b. QuickSort(A, low, $p-1$)
   c. QuickSort(A, $p+1$, $high$)
2. Return A

Note: Partition rearranges elements and returns pivot index Elements $\leq$ pivot on the left, $>$ pivot on the right

Sorting Algorithms – Summary Overview

Performance

Algorithm	Time (Avg)
Selection	$\mathcal{O}(n^2)$
Insertion	$\mathcal{O}(n^2)$
Bubble	$\mathcal{O}(n^2)$
Merge	$\mathcal{O}(n \log n)$
Quick	$\mathcal{O}(n \log n)$

Algorithm	Space
Selection	$\mathcal{O}(1)$
Insertion	$\mathcal{O}(1)$
Bubble	$\mathcal{O}(1)$
Merge	$\mathcal{O}(n)$
Quick	$\mathcal{O}(\log n)$

Pros / Cons

Algorithm	Advantage
Selection	Simple, few swaps
Insertion	Fast on nearly sorted
Bubble	Easy to code
Merge	Always fast, stable
Quick	Very fast in practice

Algorithm	Disadvantage
Selection	Slow
Insertion	Still $\mathcal{O}(n^2)$
Bubble	Very inefficient
Merge	Uses extra space
Quick	Worst case is $\mathcal{O}(n^2)$